# DISEÑO SOSTENIBLE DE UN SERVICIO TIC CASO DE ESTUDIO: AUFGABE HOME

## INTRODUCCIÓN

Empresa analizada: AUFGABE HOME

Proyecto: Plataforma web y móvil de gestión empresarial para clientes corporativos.

La transformación digital suele asociarse a conceptos cono la innovación, productividad o automatización. Sin embargo, raramente se analiza el coste oculto que existe detrás de cada petición de HTTP, cada gigabyte transferido o cada proceso ejecutado de forma innecesaria. En muchas ocasiones, la degradación de la eficiencia no se percibe de manera inmediata, pero sus consecuencias terminan acumulándose en forma de mayor consumo energético, infraestructuras sobredimensionadas y una vida útil mas corta del hardware.

En este contexto, Aufgabe Home pretende lanzar una plataforma web y móvil de gestión empresarial capaz de procesar información en tiempo real, integrar contenido multimedia de alta resolución y mantener una comunicación continua con servicios externos mediante APIs. El objetivo de este informe es analizar los impactos ambientales y sociales derivados de estas características y proponer medidas de optimización basadas en los principios del Green Software Engineering, Green Coding y del diseño inclusivo.

## PARTE 1 – IDENTIFICACIÓN DE IMPACTOS

### 1.- IMPACTOS AMBIENTALES DIRECTOS E INDIRECTOS

#### IMPACTOS AMBIENTALES DIRECTOS

Los impactos ambientales directos son aquellos que se producen durante la ejecución normal del software.

#### ELEVADO CONSUMO COMPUTACIONAL

El procesamiento de datos en tiempo real exige una utilización constante de CPU, memoria y recursos de red. Cuanto mayor sea la carga de procesamiento, mayor será también el consumo eléctrico tanto en dispositivos cliente como en servidores.

#### TRANSFERENCIA MASIVA DE DATOS

La utilización de imágenes, vídeos y recursos multimedia de alta resolución incrementa significativamente el tráfico de red. Aunque una única transferencia pueda parecer irrelevante, millones de solicitudes diarias generan un impacto energético considerable.

#### POLLING CONTINUO HACIA APIS EXTERNAS

La realización de consultas periódicas, independientemente de que existan cambios en los datos, provoca una actividad permanente en la infraestructura. Esto aumenta el número de peticiones procesadas y eleva innecesariamente el consumo energético global del sistema.

#### SOBRECARGA DERIVADA DE DEPENDENCIAS INNECESARIAS

La incorporación de frameworks y librerías sobredimensionadas puede aumentar el tiempo de ejecución, el uso de memoria y el tamaño final de la aplicación, reduciendo su eficiencia operativa.

#### IMPACTOS AMBIENTALES INDIRECTOS

Los impactos indirectos aparecen a medio y largo plazo como consecuencia del funcionamiento del sistema.

### INCREMENTO DE LA HUELLA DE CARBONO DIGITAL

Todo recurso computacional requiere energía. Cuando el consumo de CPU, almacenamiento y transferencia de datos aumenta, también lo hacen las emisiones asociadas a la producción de esa energía.

### SOBREDIMENSIONAMIENTO DE INFRAESTRUCTURA

Una aplicación poco optimizada requiere más capacidad de procesamiento, más almacenamiento y más recursos cloud, generando costes económicos y ambientales adicionales.

### OBSOLESCENCIA TECNOLÓGICA INDUCIDA

Cuando una aplicación exige cada vez más recursos hardware para funcionar correctamente, muchos usuarios terminan sustituyendo sus dispositivos antes de tiempo.

### GENERACIÓN DE RESIDUOS ELECTRÓNICOS

La renovación prematura de dispositivos contribuye directamente al aumento de residuos electrónicos, uno de los principales desafíos ambientales de la industria tecnológica actual.

## 2.- IMPACTOS SOCIALES Y DE ACCESIBILIDAD

La sostenibilidad no se limita al medio ambiente. Un producto digital también debe ser socialmente sostenible.

### BRECHA DIGITAL

Si la plataforma requiere dispositivos modernos o conexiones de alta velocidad, determinados grupos de usuarios pueden quedar excluidos del acceso al servicio.

### BARRERAS DE ACCESIBILIDAD

Una interfaz que no contemple estándares de accesibilidad puede resultar inutilizable para personas con discapacidad visual, auditiva o motora.

### EXCLUSIÓN POR COMPLEJIDAD TECNOLÓGICA

Los sistemas excesivamente complejos generan dificultades para usuarios con menor alfabetización digital, limitando la igualdad de acceso a la información.

### DEPENDENCIA TECNOLÓGICA

Cuando el software obliga constantemente a actualizar equipos o sistemas operativos, se incrementa la dependencia económica de los usuarios respecto al fabricante del hardware.

### DETERIORO DE LA EXPERIENCIA DE USUARIO

Tiempos de carga elevados, bloqueos frecuentes o consumo excesivo de batería generan frustración y reducen la percepción de calidad del servicio.

# PARTE 2 – PROPUESTA DE MEDIDAS

| Nº | Medida propuesta | Tipo (ambiental/social) | Justificación técnica breve |
|---|---|---|---|
| 1 | Sustituir el ***polling*** continuo por ***WebSockets*** o eventos en tiempo real. | Ambiental | Permite transmitir información únicamente cuando se producen cambios, eliminando miles de peticiones innecesarias. |
| 2 | Aplicar compresión multimedia y formatos optimizados como o ***AVIF***, junto con ***lazy loading***. | Ambiental | Reduce la cantidad de datos transferidos y mejora los tiempos de carga. |
| 3 | Auditar y minimizar dependencias externas. | Ambiental | Disminuye el tamaño del proyecto, reduce la deuda técnica y mejora la eficiencia de ejecución. |
| 4 | Aplicar las pautas de accesibilidad ***WCAG*** universales (contraste adecuado, navegación por teclado, etiquetas semánticas, etc.). | Social | Garantiza la utilización del sistema por personas con distintas capacidades funcionales. |
| 5 | Diseñar una interfaz progresiva y adaptable a hardware modesto. | Ambiental y social | Prolonga la vida útil de los dispositivos y reduce la exclusión tecnológica. |

# PARTE 3 – REFLEXIÓN FINAL

## 1.- ¿QUÉ MEDIDA DE OPTIMIZACIÓN DE SOFTWARE CONSIDERAS MÁS PRIORITARIA EN ESTE ESCENARIO Y POR QUÉ?

Considero que la sustitución del ***polling*** continuo por una arquitectura basada en eventos es la medida más prioritaria. El motivo es que afecta simultáneamente al cliente, al servidor y a la red. Una consulta innecesaria puede parecer insignificante, pero cuando se multiplica por miles de usuarios y millones de solicitudes termina convirtiéndose en un consumo energético permanente y difícil de justificar técnicamente.

La mejor optimización suele ser aquella que elimina trabajo innecesario antes de que llegue a ejecutarse.

## 2.- ¿CÓMO INFLUYE EL DESARROLLO DE SOFTWARE EFICIENTE (GREEN CODING) EN EL COSTE DE INFRAESTRUCTURA EN LA NUBE Y EN LA EXPERIENCIA DE USUARIO (UX) DE UNA APLICACIÓN?

El Green Coding persigue la misma meta que cualquier estrategia de optimización seria: realizar más trabajo utilizando menos recursos.

Una aplicación eficiente necesita menos instancias cloud, menos almacenamiento y menos capacidad de procesamiento, lo que reduce directamente los costes operativos. Desde el punto de vista del usuario, la consecuencia es igualmente positiva: menores tiempos de carga, menor consumo de batería y una aplicación más fluida y estable.

Por tanto, la eficiencia no debe verse únicamente como una cuestión medioambiental, sino también como una mejora económica y funcional.

### 3.- ¿QUÉ PAPEL JUEGA EL DESARROLLADOR/A DE SOFTWARE EN LA PREVENCIÓN DE LA OBSOLESCENCIA PROGRAMADA DEL HARDWARE?

Existe la tendencia a considerar que la obsolescencia es únicamente responsabilidad de los fabricantes de dispositivos. Sin embargo, el software también puede convertirse en un factor determinante.

Cada vez que un desarrollador añade complejidad innecesaria, dependencias sobredimensionadas o procesos ineficientes, está elevando los requisitos de hardware de la aplicación. Por el contrario, un desarrollo optimizado permite que dispositivos antiguos continúen siendo funcionales durante más tiempo.

En consecuencia, el desarrollador tiene una responsabilidad directa en la prolongación del ciclo de vida tecnológico y en la reducción de residuos electrónicos.

## CONCLUSIÓN

Cuando se habla de sostenibilidad tecnológica solemos derivarlo directamente con los centros de datos, en las energías renovables o en las infraestructuras cloud. Sin embargo, sabiendo que problema nace mucho antes, en decisiones aparentemente pequeñas tomadas durante la fase de desarrollo.

En el caso de AUFGABE HOME, el procesamiento intensivo de datos, la transferencia continua de información y el uso indiscriminado de componentes externos pueden derivar en un ecosistema digital menos eficiente, más costoso y ambientalmente más agresivo. Ninguna de estas decisiones resulta crítica por sí sola, pero su acumulación genera un impacto futuro y muy evidente cuando la aplicación escala.

La sostenibilidad en el software no consiste en reducir prestaciones, sino en eliminar los excedentes computacionales. Tener un código más eficiente consume menos recursos, por lo que mejora la experiencia de usuario, reduce costes operativos y prolonga la vida útil del hardware existente.

Como desarrolladores, no solo construimos aplicaciones. También definimos cuántos recursos van a consumir, cuánto hardware será necesario para ejecutarlas y cuál será su impacto social. Por ello, diseñar software sostenible no debería considerarse una característica adicional, sino una manifestación de calidad técnica y responsabilidad profesional.